4.2 密度聚类

K-Means的不足 & DBSCAN聚类

前情回顾

K-Means算法步骤

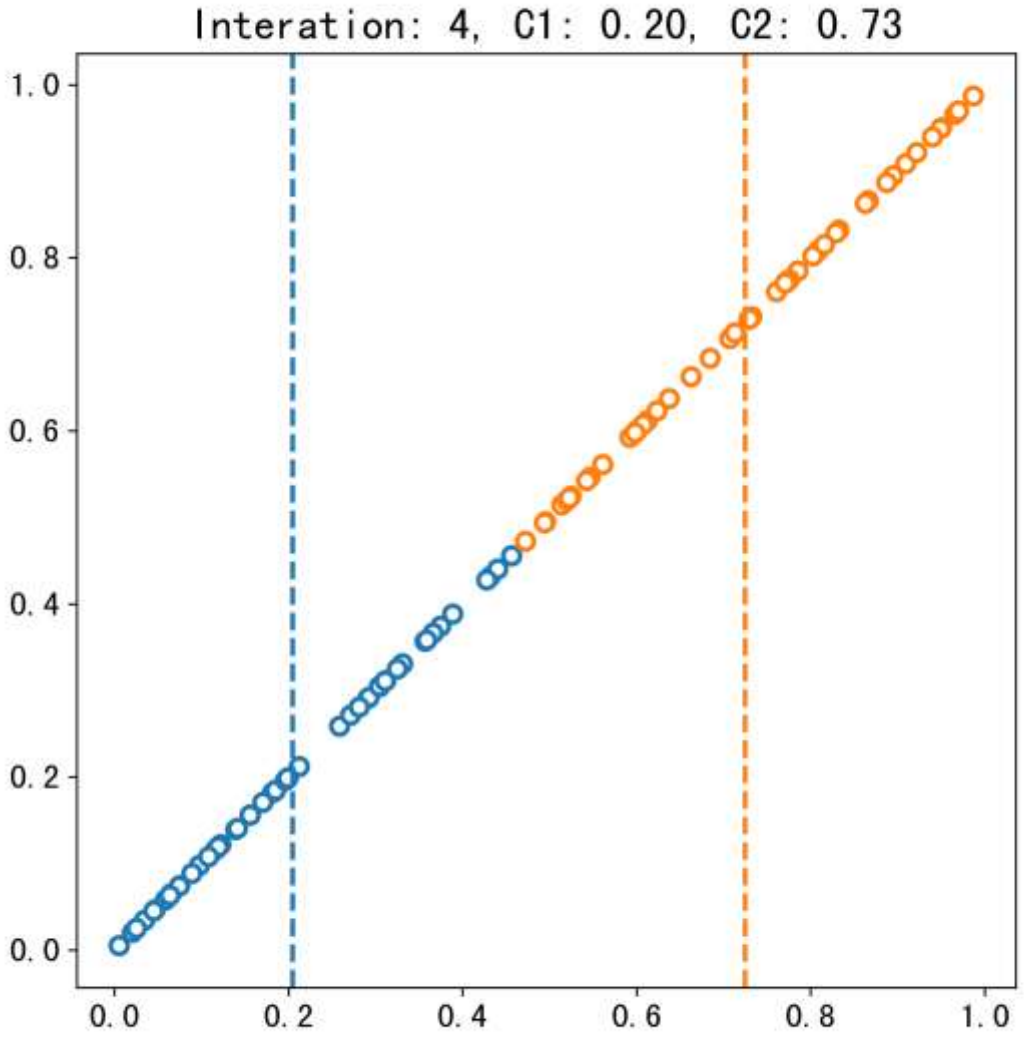
① 初始化k和中心点位置

循环以下步骤至中心点位置稳定:

② 计算所有点到k个中心点的位置

③ 将各个点与最近中心点归为一类

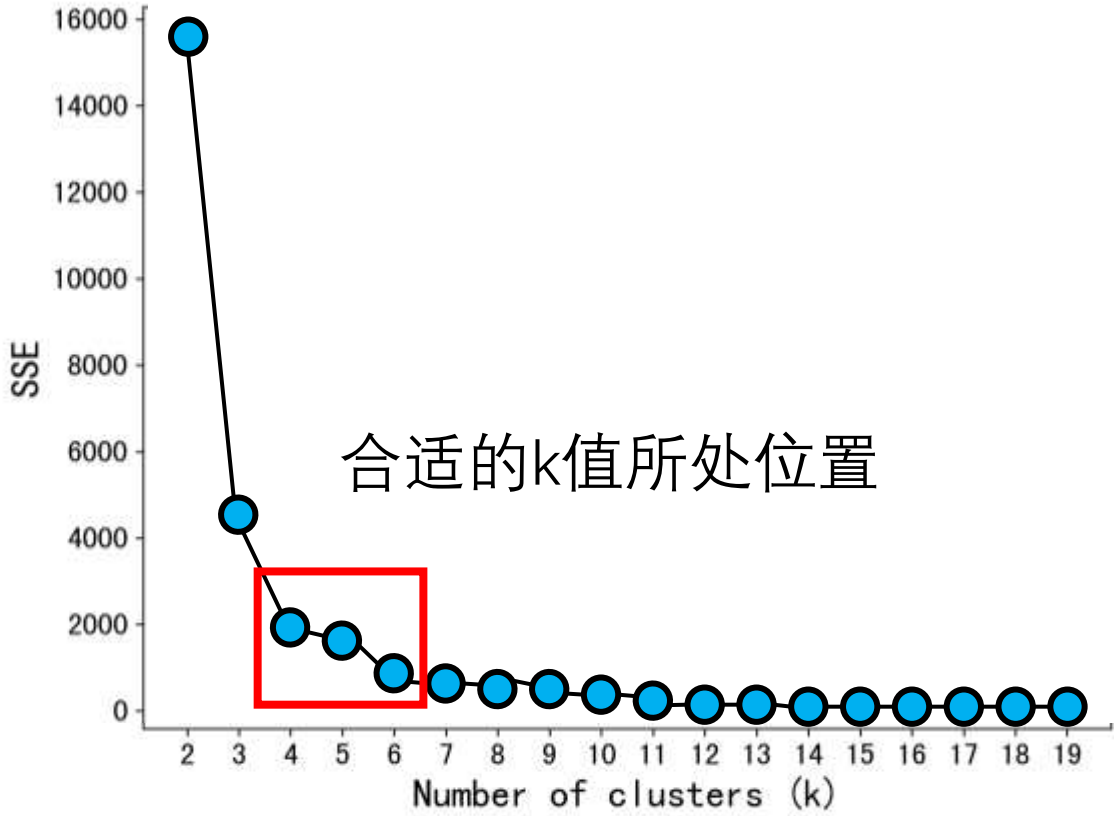
④ 逐一计算各簇的均值并作为新中心点

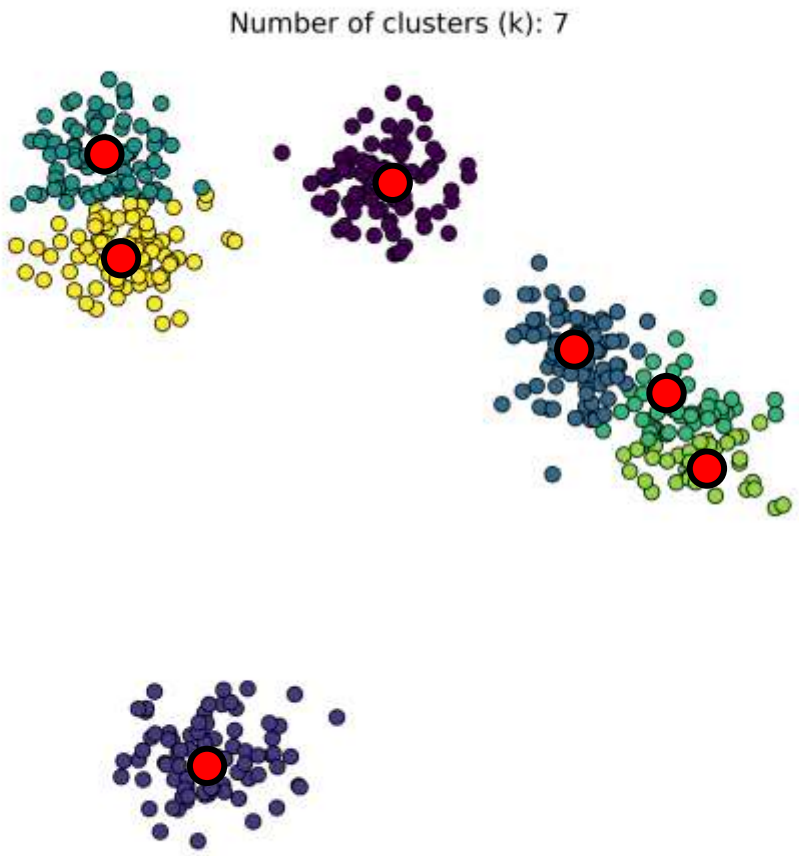


前情回顾

使用K和SSE画出肘部曲线（Elbow Curve）

尝试不同的K值（2，…，19）



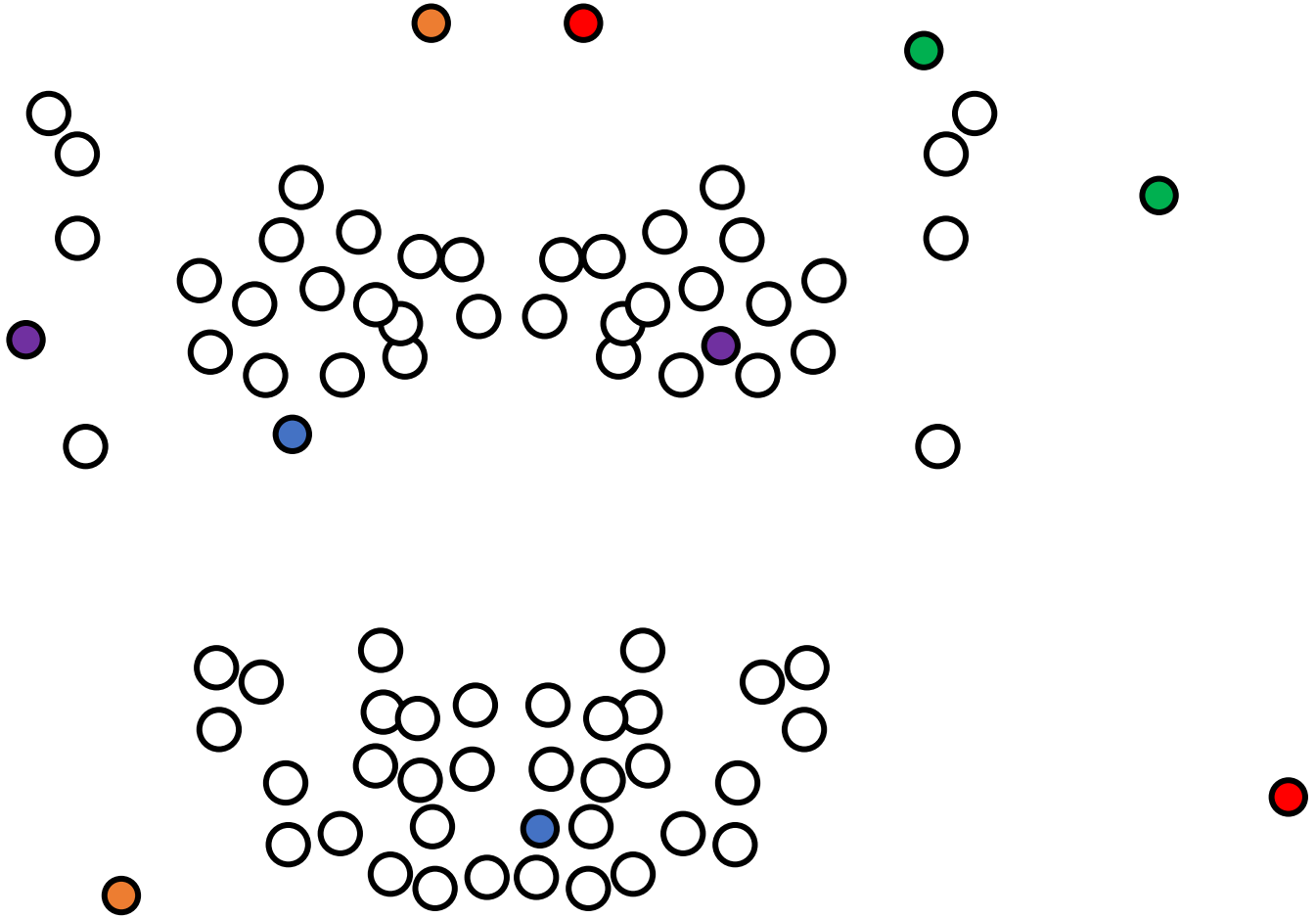


前情回顾

中心点的初始位置也会影响聚类效果

一种特殊的情况

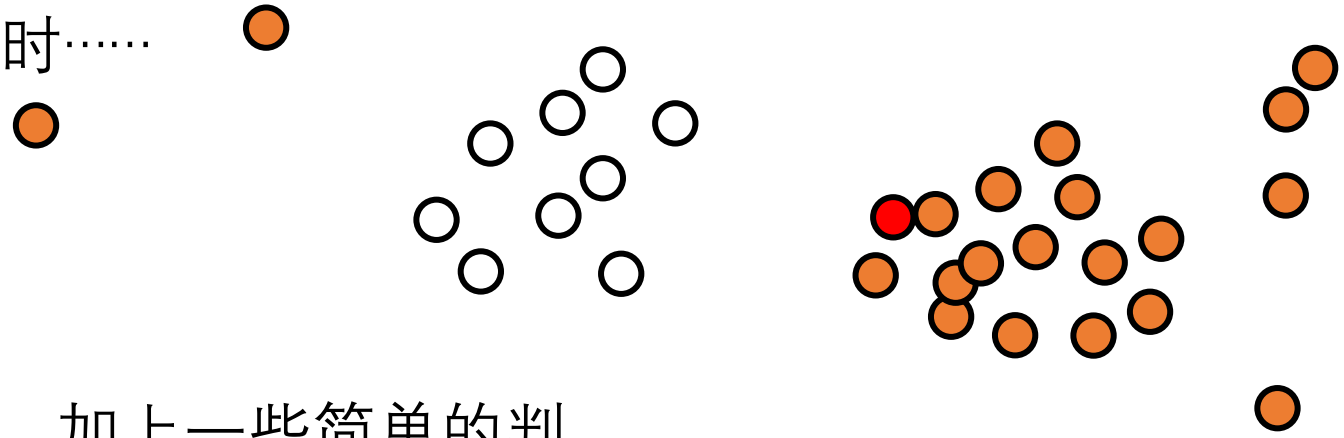
多次随机选择最优



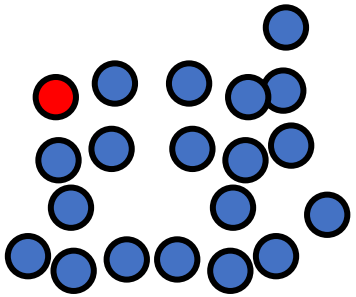
前情回顾

以靠近均值的实际样本作为中心点

当存在噪声时……



加上一些简单的判断就能检测出噪声或者新的聚类

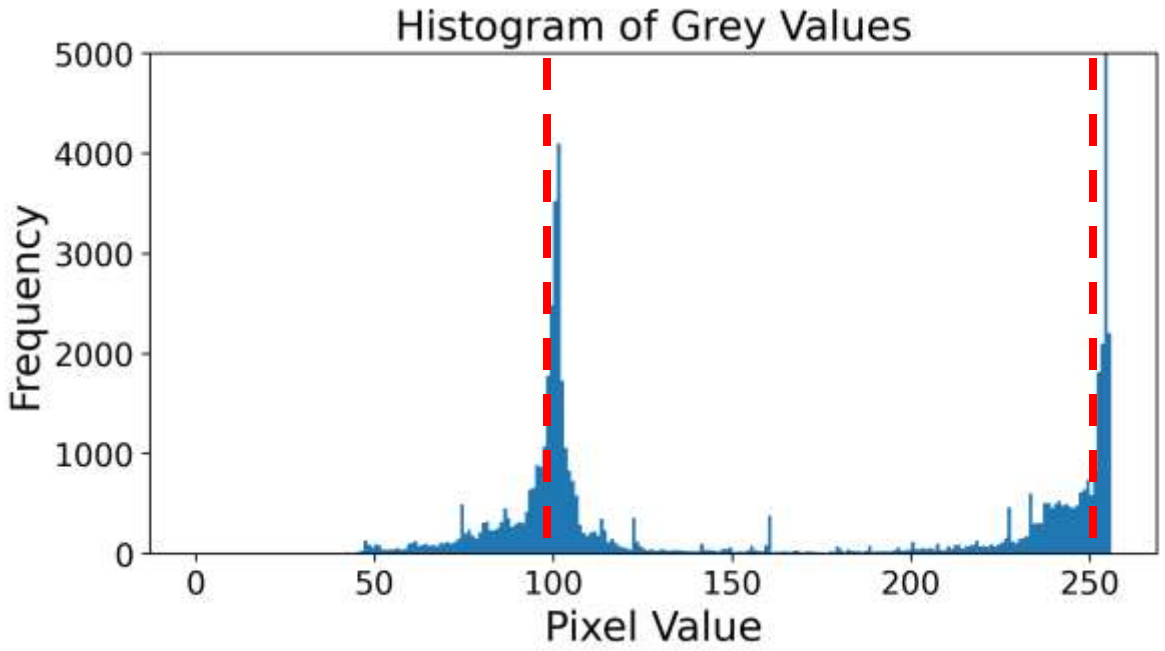


前情回顾

以车牌识别为例

① 将图片转为灰度图像：像素值在[0, 255]区间内

② 使用k-Means聚为两类



Final Centers: [252.55062255 98.44364715], Iteration:

前情回顾

K = 2

2160X1080的RGB图像

假设每个像素需要3个Byte存储

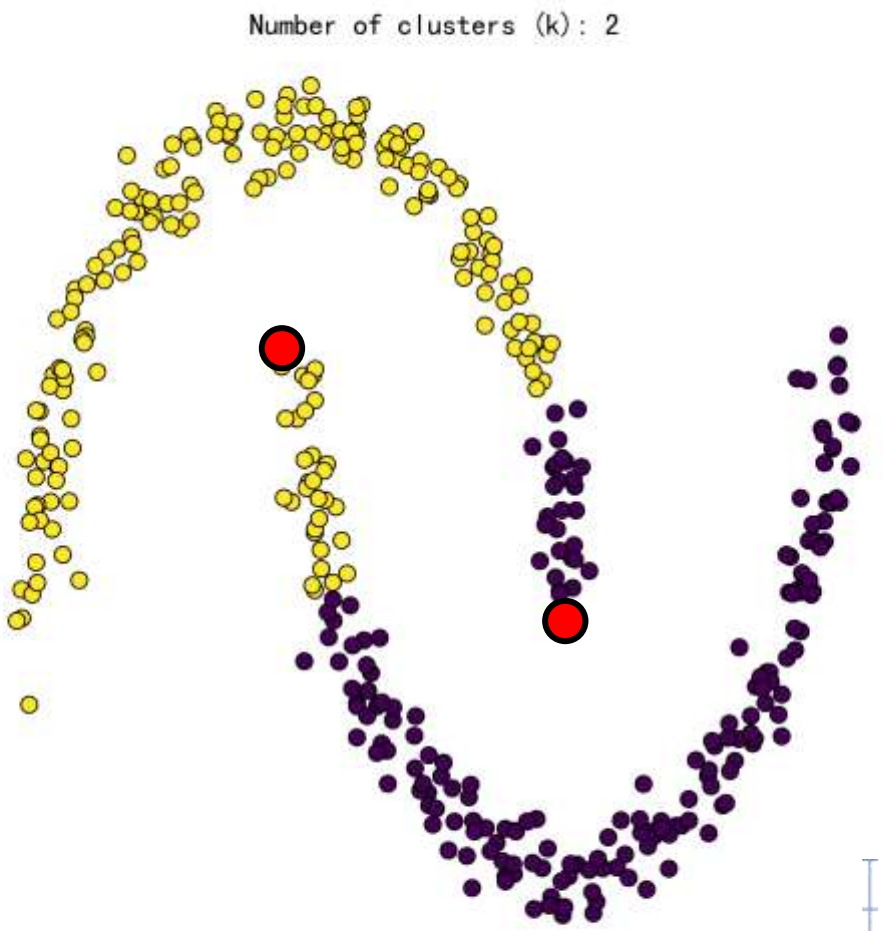
Buffer = 2160 X 1080 X 3 = 6998400 bytes

= 6998400 / 1024^2 = 6.67 MB

通过“颜色+位置”的方式编码来压缩图像

通过减少颜色进行有损下所

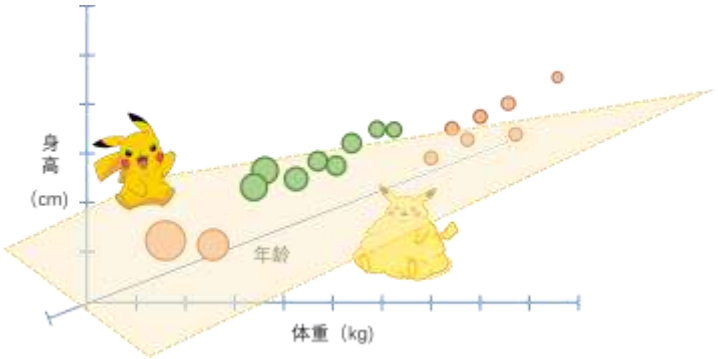
K = 4



线性不可分的情况

K-Means 难以找到合适的中心点

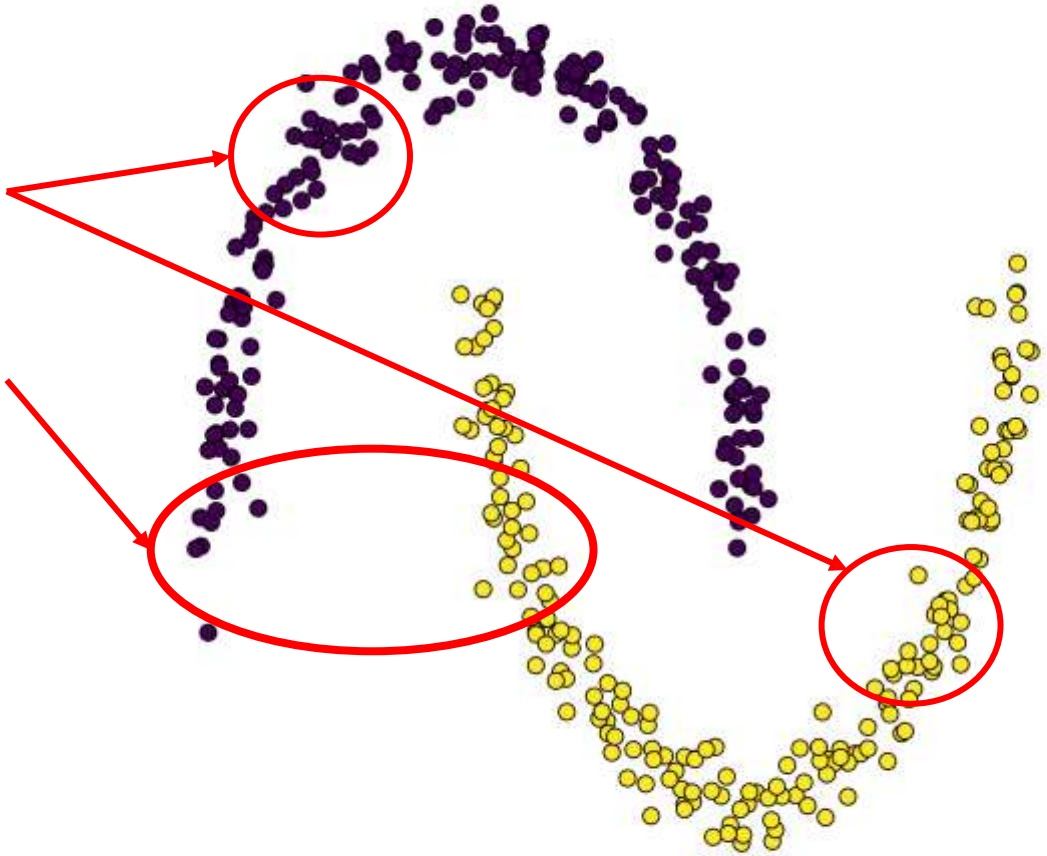
可以使用核函数的策略



有没有更加直观的方法?

同一类的样本“抱得更紧”

不同类的样本“分得更开”



给一个更正式的定义**密度**

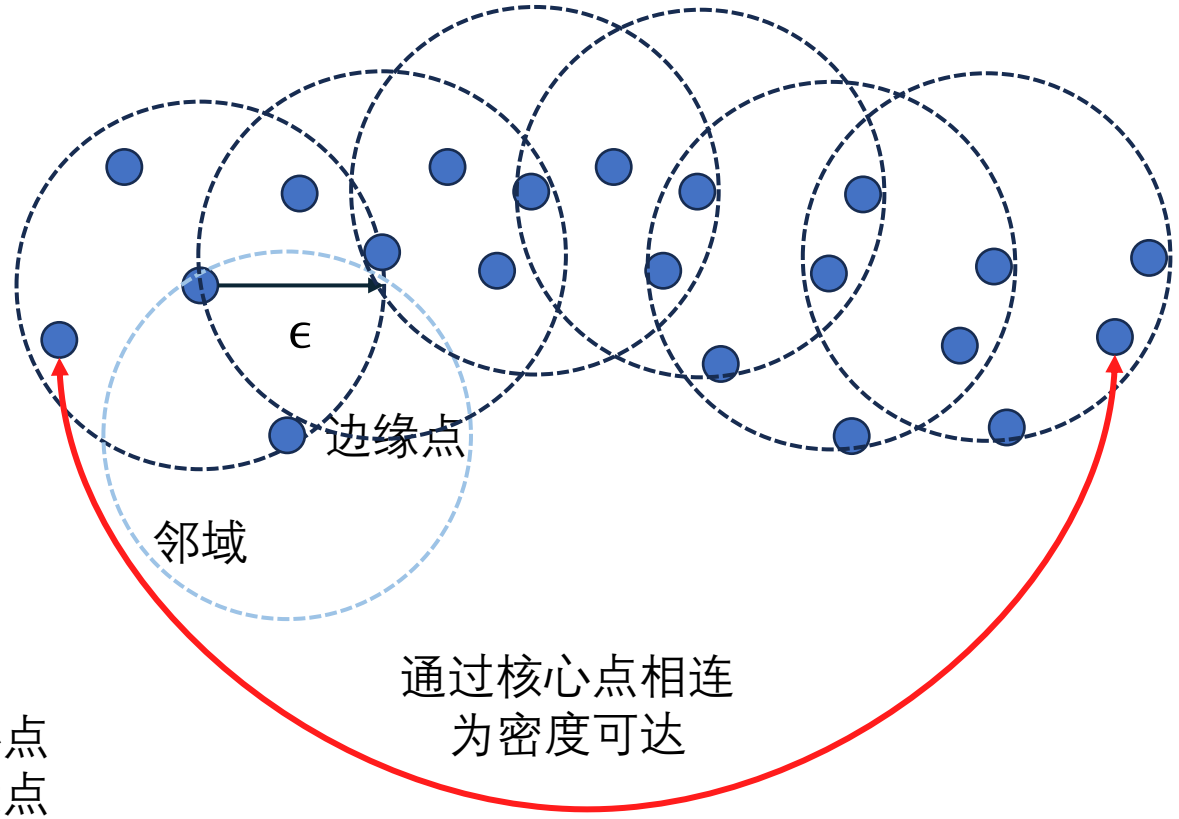
根据样本间的密度进行聚类：**DBSCAN**

***D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise*

DBSCAN

邻域半径 ϵ	最少点数 $minPts$	核心点 Core Points	边缘点 Boarder Points	离群点 Outliers	密度直达 Directly Density-reachable	密度可达 Density-reachable	密度相连 Density-connected
主要参数		$N_{cp} \geq minPts$	在核心点邻域内 $N_{cp} < minPts$		邻域内的点与核心点	通过核心点相连	对于任意两个点，存在另一点，与它们密度可达

令 $minPts = 3$



既不是核心点
也不是边缘点

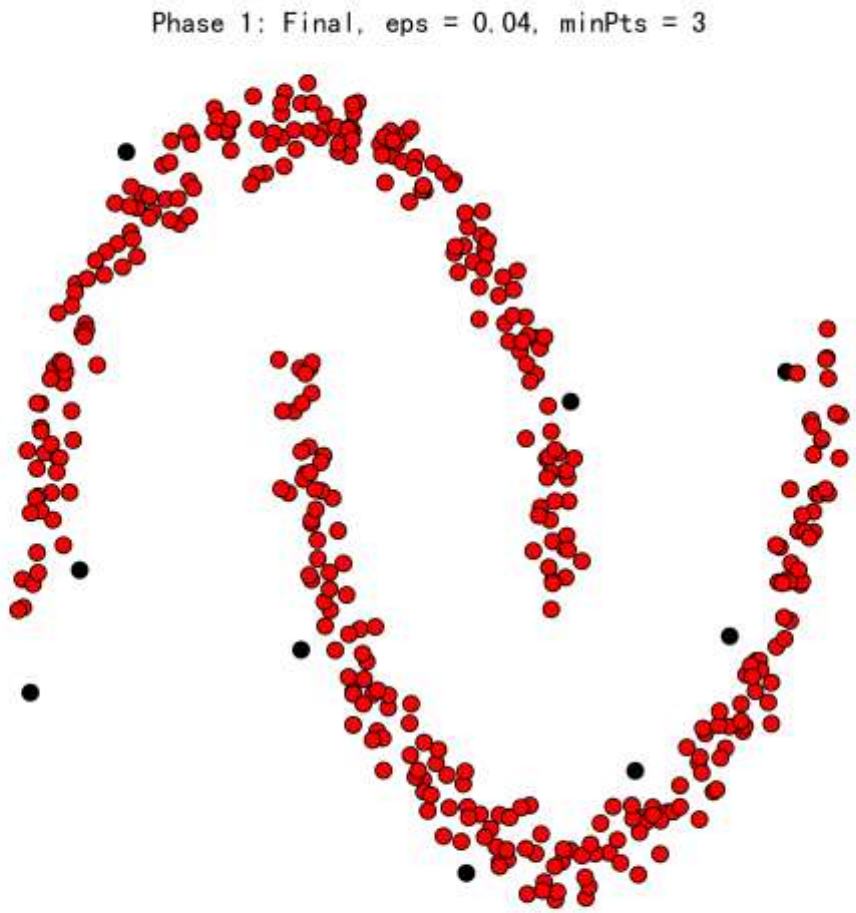
DBSCAN

主要思想：找出所有密度相连的集合

① 初始化ϵ 和$\boxed{minPts}$

特征数量的对数 $\log_2 N_{feature}$

② 遍历所有点，判断是否为核心点



DBSCAN

主要思想：找出所有密度相连的集合

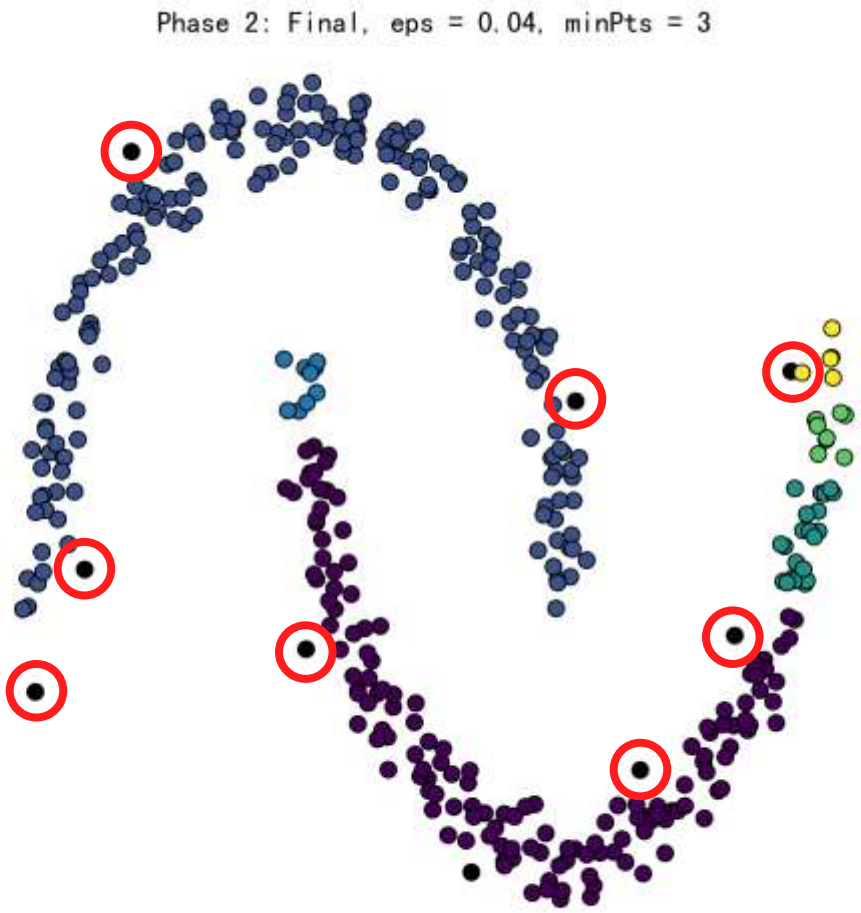
① 初始化ϵ 和$\boxed{minPts}$

特征数量的对数 $\log_2 N_{feature}$

② 遍历所有点，判断是否为核心点

③ 从随机核心点开始，确定密度相连集合

④ 重复③至所有核心点被处理完



DBSCAN

主要思想：找出所有密度相连的集合

① 初始化ϵ 和$\boxed{minPts}$

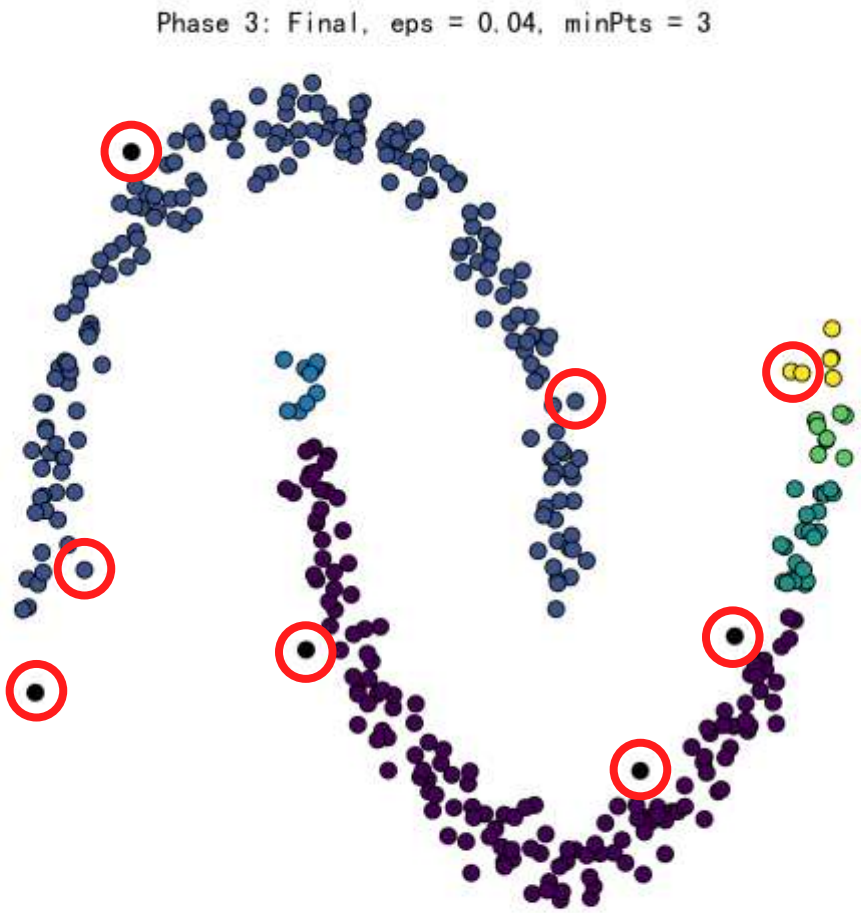
特征数量的对数 $\log_2 N_{feature}$

② 遍历所有点，判断是否为核心点

③ 从随机核心点开始，确定密度相连集合

④ 重复③至所有核心点被处理完

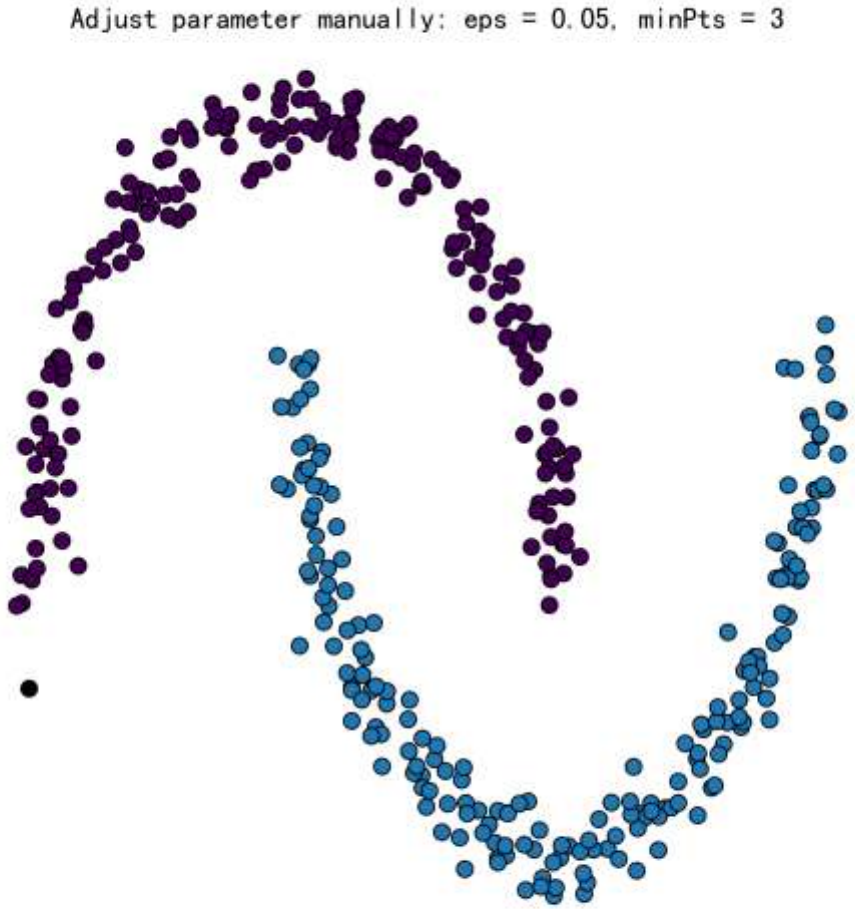
⑤ 判断剩下点的类型：噪声或边缘点

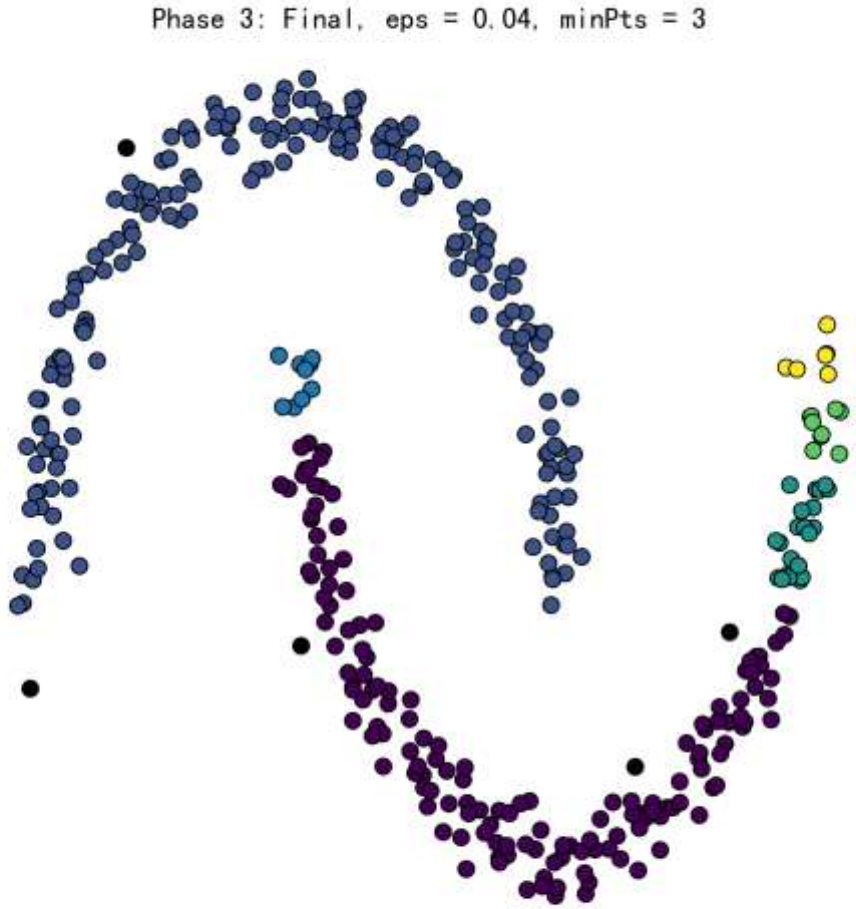


邻域半径$\epsilon = 0.04$，簇数量过多

DBSCAN —— ϵ 的选择

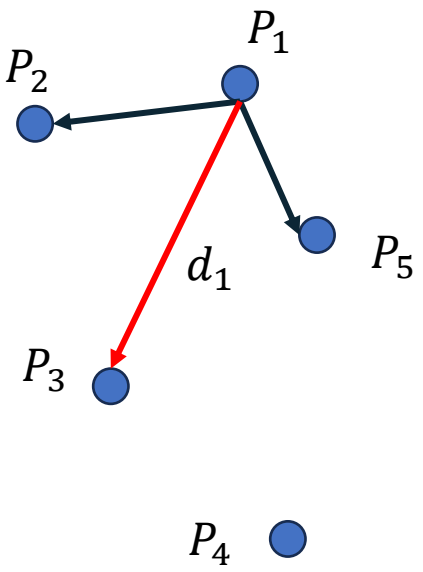
当$\epsilon = 0.05$ 时，聚类结果为……　应该如何确定 ϵ？





DBSCAN —— k-Distance

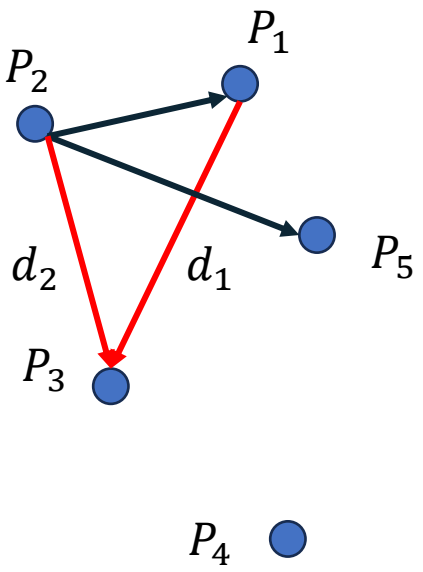
$k = minPts$，假设 $k = 3$



① 计算 P_1 到距其第 3 近之点的距离 d_1

DBSCAN —— k-Distance

$k = minPts$，假设 $k = 3$

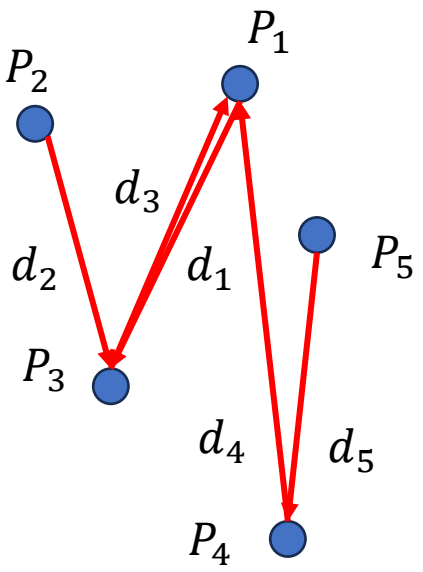


① 计算 P_1 到距其第 3 近之点的距离 d_1

② 计算 P_2 到距其第 3 近之点的距离 d_2

DBSCAN —— k-Distance

$k = minPts$，假设 $k = 3$



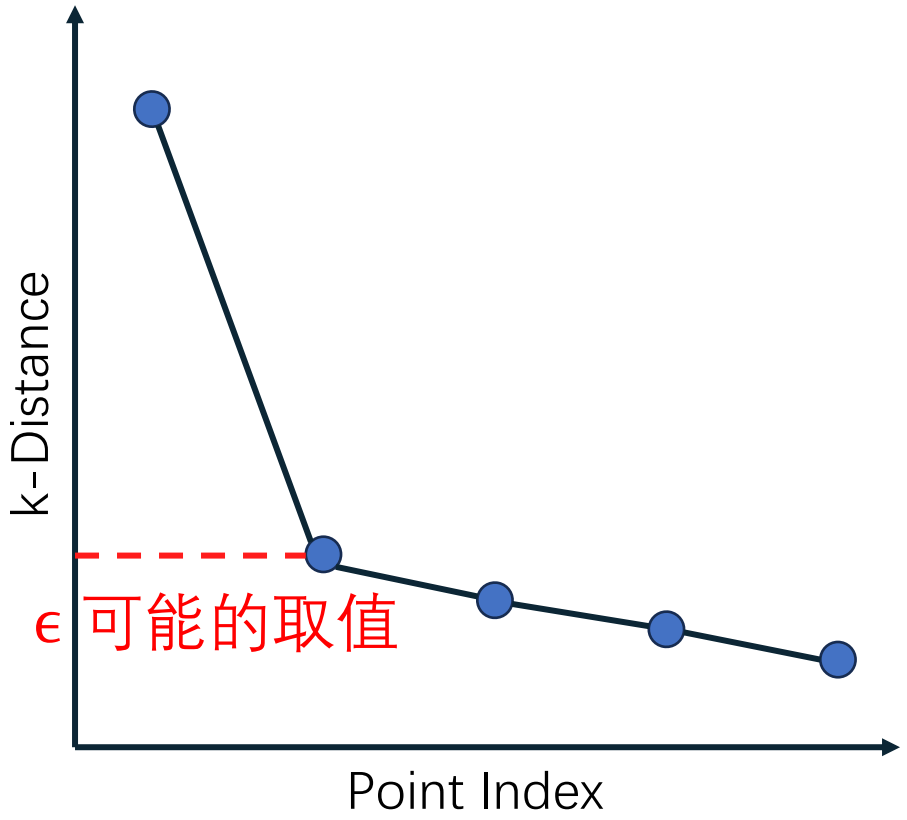
① 计算 P_1 到距其第 3 近之点的距离 d_1

② 计算 P_2 到距其第 3 近之点的距离 d_2

③ 计算 P_3 到距其第 3 近之点的距离 d_3

④ 计算 P_4 到距其第 3 近之点的距离 d_4

⑤ 计算 P_5 到距其第 3 近之点的距离 d_5

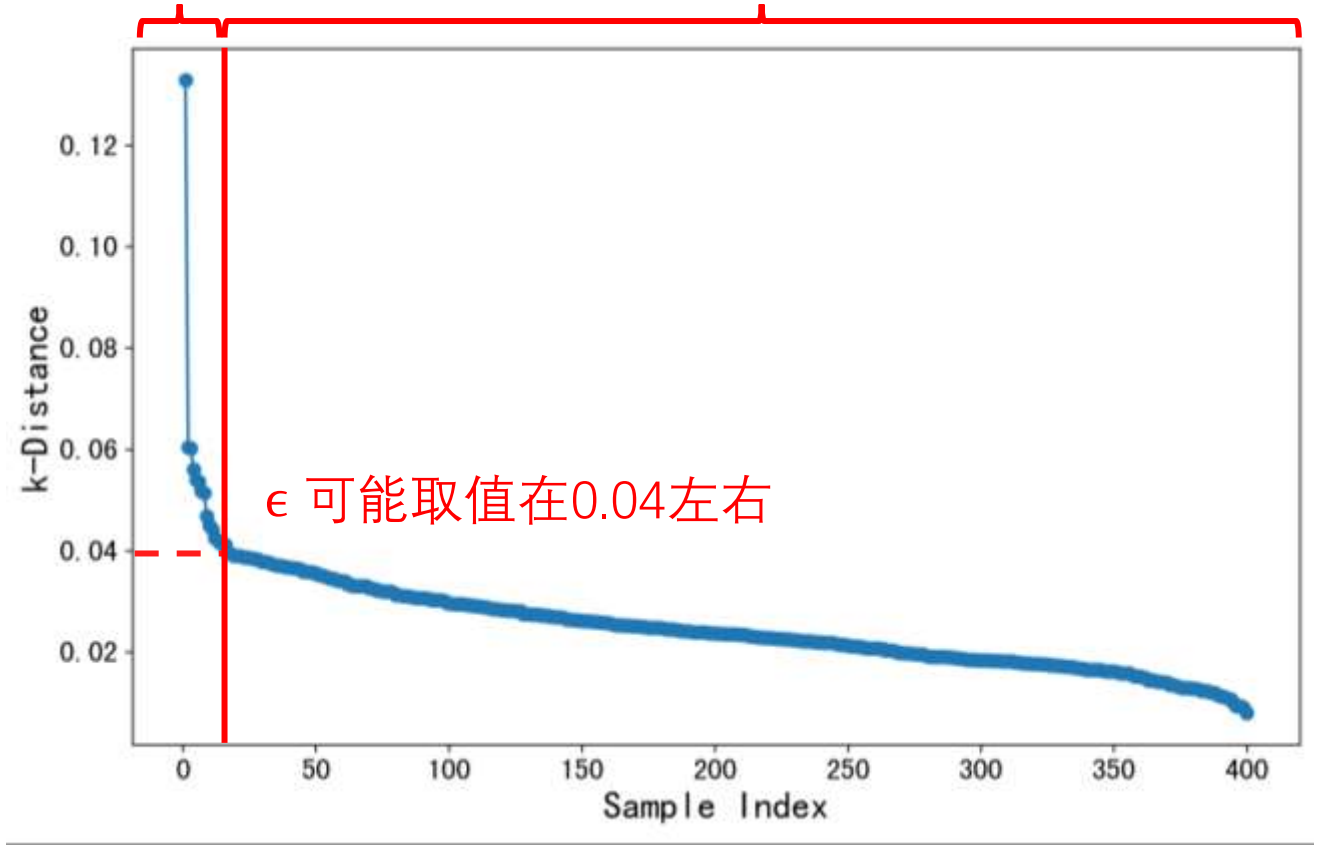


对 d_1, d_2, d_3, d_4, d_5 排序，形成的曲线即为k-Distance曲线

DBSCAN —— k-Distance

极少点距离大于ϵ
可能是异常值

大多数点距离小于ϵ都可能
密度相连，形成聚类

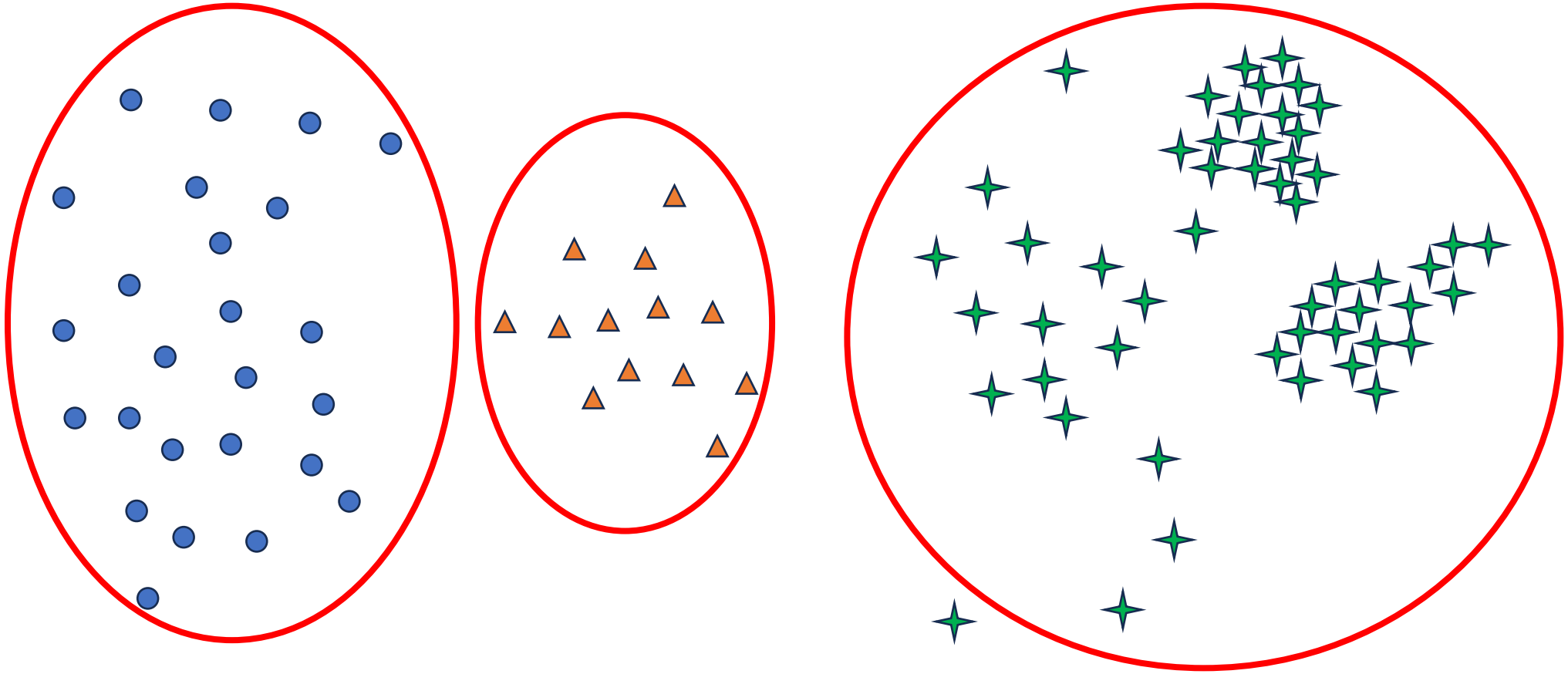


K-Means vs DBSCAN

特性	K-Means	DBSCAN
聚类数	需要预先指定	自动发现簇数量
适用数据形状	只适合球状或凸形簇	适合任意形状的簇
对噪声的敏感性	对噪声敏感	对噪声鲁棒
计算复杂度	O(n * k * d)，较快	O(n * log(n))，可能较慢
对初始值敏感性	对初始质心敏感	不敏感
参数设置	需要指定簇的数量	需要设定eps和MinPts
适合大数据集	适合大数据集	对大数据集可能较慢
高维数据效果	较好	较差，尤其是高维空间中的距离度量不可靠

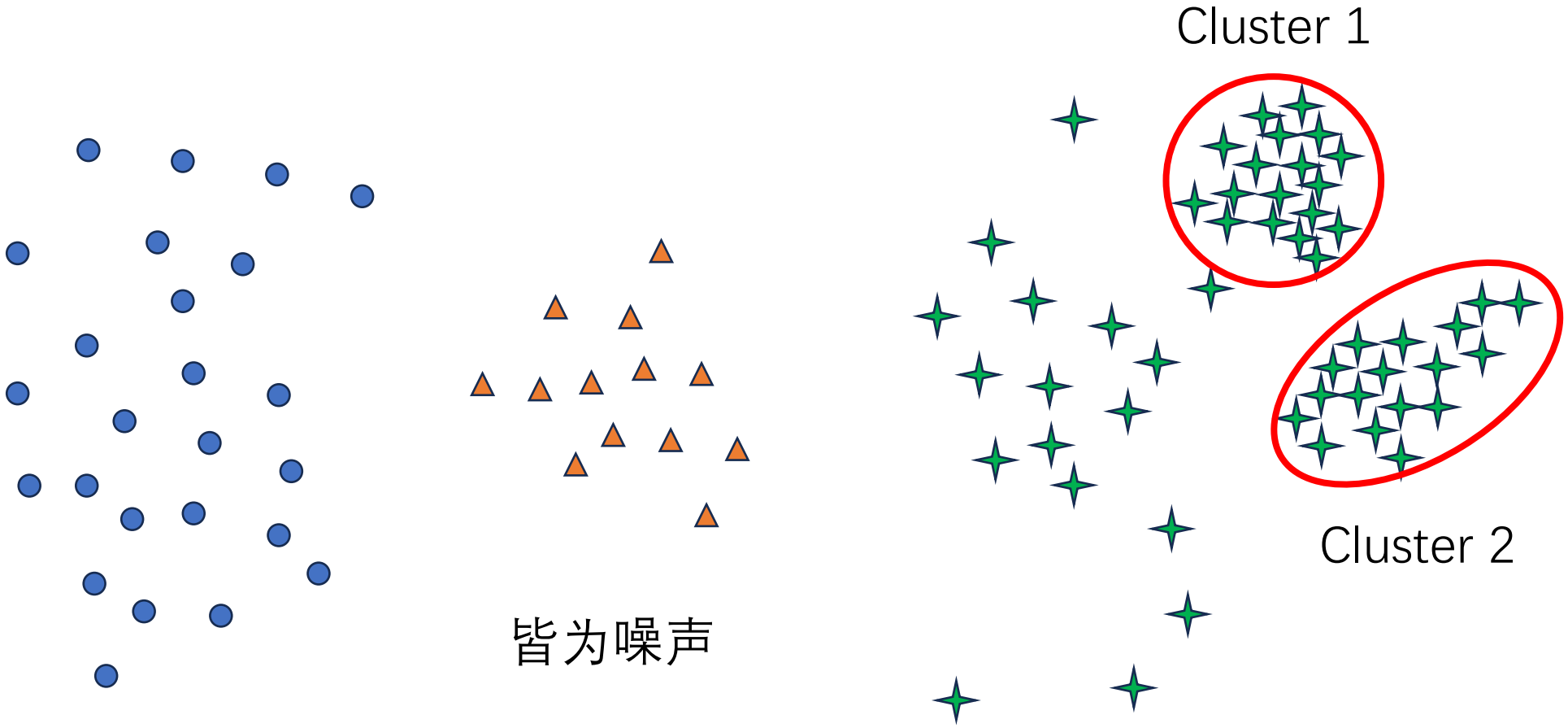
DBSCAN不适用密度变化大的情况

理想聚类结果……



DBSCAN不适用密度变化大的情况

实际聚类结果……



小结

回顾K-Means算法及不足

DBSCAN概念及算法

K-Means vs DBSCAN